Hafta 4 — Derste Yapılacak Uygulamalar

> **Ortam:** Sınıfta öğrenci bilgisayarı **yok**. Tüm uygulamalar **hoca ekranında canlı** yapılır; öğrenci tahmin eder, hoca çalıştırıp doğrular. Kodlama öğrenciye **ev görevi** olarak devredilir. **Toplam süre:** 32 dakika — blok sürelerinin **içindedir** (bkz. `H04-anlatim.md` bölüm 2) **Gerekli:** Projeksiyon, tarayıcı, **F12 geliştirici araçları**, `Hafta-04/KampusKitap`.

🔧 Hazırlık — Derse girmeden önce (5 dk, ders dışı)

- ☐ `dotnet run` ile proje açıldı; `/Form`, `/Form/Kaynak/ROTA?deger=SORGU`, `/Kitap/Ekle` sayfaları bir kez denendi
- ☐ **F12 → Network** sekmesi açık bırakıldı; bir form gönderilip **Payload** bölümünün göründüğü doğrulandı
- ☐ Tarayıcı yakınlaştırması **125–150%** (Network panelindeki yazılar küçük)
- ☐ Terminal ve tarayıcı **yan yana**
- ☐ `H04-etkilesimli.html` açıldı, N denendi ve **kapatıldı**
- ☐ Slayt 14'teki 2B sahne bir kez adım adım ilerletildi

> 💡 **Zaman kazancı:** Bu haftanın bütün deney sayfaları projede **hazır**. Derste sıfırdan kod yazmak yerine hazır sayfayı açıp **sonucu** tartışıyoruz; yalnızca Uygulama C'de tek satır canlı değişiyor.

Uygulama A — Formu elle yaz, gövdeyi gör

Süre: 10 dakika · **Blok:** A **Amaç:** Hafta 3'ün kapanış sorusunu kanıtla cevaplamak: tarayıcı gövdeye ne koyuyor?

Adım 1 — Soruyu topla (1 dk)

Tahtada duran geçen hafta sorusunu okuyun:

> *"Bir form 'Gönder' dendiğinde tarayıcı isteğin gövdesine tam olarak ne koyar?"*

> **Sınıfa sor:** tahminleri **tahtaya yazın**, silmeyin. Birkaç dakika sonra üstüne gideceğiz.

Beklenen cevaplar: "JSON", "alan adlarıyla birlikte", "şifrelenmiş olarak". Hiçbirini düzeltmeyin.

Adım 2 — `/Form` sayfasını aç ve gönder (4 dk)

`http://localhost:5xxx/Form`

Formda beş alan var. İkisi özel:

Alan	Özelliği
`id="notAlani"` · `name="Notlar"`	`id` ile `name` **bilerek farklı**
`id="unutulan"` · `name` **YOK**	Hiç gönderilmeyecek

Gönder'e basın. Sayfa isteğin gövdesini **olduğu gibi** basıyor:

```
Baslik=Suc+ve+Ceza&Yazar=Dostoyevski&Fiyat=120&Notlar=deneme
```

> **Tahtaya dönün.** Tahminlerle karşılaştırın. JSON değil; `ad=değer&ad=değer`.

Adım 3 — Üç gözlemi tek tek okutun (3 dk)

> **Sınıfa sor:** "*Gövdede `notAlani` mı var, `Notlar` mı?*" Cevap: **`Notlar`** — yani `name`.

> **Sınıfa sor:** "*`unutulan` alanının değeri nerede?*" Cevap: **hiçbir yerde.** `name` yoksa alan gönderilmez.

Sonra `Content-Type` satırını gösterin: `application/x-www-form-urlencoded`. Boşluğun `+` olduğunu, Türkçe karakterlerin `%XX` olduğunu gösterin.

⚠️ **Söylenecek cümle:** "*Dönem boyunca 'formum boş geliyor' diyeceksiniz. Cevap neredeyse her zaman bu iki satırda: `name` yok ya da yanlış yazılmış.*"

Adım 4 — Aynı şeyi tarayıcıdan doğrula (2 dk)

F12 → Network sekmesini açın, formu yeniden gönderin, listedeki `Form` isteğine tıklayın → **Payload** (ya da **Request**) bölümü.

Aynı gövde, bu kez **tarayıcının kendi aracında**.

> **Söylenecek cümle:** "*Bu paneli bu dönem çok kullanacaksınız. 'Acaba ne gönderdim?' sorusunun cevabı tahminde değil, burada.*"

Yedek plan: Network paneli projeksiyonda okunmuyorsa yalnızca sayfadaki ham gövdeyle yetinin; panel gösterimini ev görevine bırakın.

Uygulama B — Model Binding: kaynak sırası ve Model Binding hatası

Süre: 10 dakika · **Blok:** B **Amaç:** Model Binding kaynaklarının öncelik sırasını **ezberletmeden** göstermek ve bağlanamayan değerin sessizce varsayılana döndüğünü kanıtlamak.

Adım 1 — Kaynak sırası: tahmin ettir (2 dk)

`/Form/Kaynak/ROTA?deger=SORGU` adresini açın. Sayfa üç kaynağı ayrı ayrı gösteriyor:

Kaynak	Değer
Rota	`ROTA`
Query string	`SORGU`
Form gövdesi	`(yok)`

Kazanan: **ROTA**.

> **Sınıfa sor (POST düğmesine basmadan):** "*Şimdi gövdede de `deger=FORM` göndereceğim. Üçü yarışacak. Hangisi kazanır?*"
>
> **El kaldırtın.** Çoğunluk genellikle **SORGU** ya da **ROTA** der.

Adım 2 — POST düğmesine bas (3 dk)

Kazanan: **FORM**.

Tahtaya sırayı yazın ve bırakın:

```
1. Form gövdesi
2. Route data
3. Query string
```

> **Söylenecek cümle:** "*Query stringni adres çubuğunda gördüğümüz için en güçlü sanıyoruz. En zayıf olan o.*"

Adım 3 — `[FromQuery]` ile sırayı boz (2 dk)

Aynı istek, farklı adres: `/Form/KaynakZorla/ROTA?deger=SORGU` (POST). Tek fark action parametresinde `[FromQuery]` olması.

Kazanan: **SORGU**.

> **Söylenecek cümle:** *"Attributeler yeni bir kaynak eklemiyor; var olan sırayı devre dışı bırakıyor."*

Adım 4 — Model Binding başarısız olunca (3 dk)

`/Form/Bagla?sayi=abc&sayiNullable=abc&tarih=xyz`

> **Sınıfa sor (açmadan):** *"Üç parametrenin üçü de çevrilemez. Ne göreceğiz?"* Beklenen cevap: "hata sayfası", "sarı ekran".

Açın:

Parametre	Gelen
`int sayi`	**0**
`int? sayiNullable`	**null**
`DateTime tarih`	**0001-01-01**
`ModelState.IsValid`	**False**

⛔ **Buradan atlamayın.** Sorun:

> "*`if (sayi == 0)` yazsam — kullanıcı sıfır mı girdi, saçma bir şey mi girdi?*"

Cevap: **ayırt edilemez.** Ayrımı yapan tek şey `ModelState`.

> **Köprü:** *"Hafta 6'nın tamamı bu nesnenin üstüne kurulu."*

Uygulama C — PRG ve over-posting

Süre: 12 dakika · **Blok:** C **Amaç:** F5'in kaydı ikinci kez eklediğini **canlı** göstermek; sonra formun HTML'ine güvenilemeyeceğini kanıtlamak.

Adım 1 — PRG'siz gönder ve F5'e bas (4 dk)

`/Kitap/Ekle` → formu doldurun → **"Kaydet (PRG'siz — yanlış)"** düğmesi.

Gelen sayfada kayıt sayısı yazıyor. Şimdi **F5**.

Tarayıcı "*Formu yeniden göndermek istiyor musunuz?*" diye sorar → **Devam et**.

Kayıt sayısı **arttı**. Bir kez daha F5 → bir daha arttı.

> **Söylenecek cümle:** *"Kullanıcı hata yapmadı. Biz adres çubuğunda bir POST bıraktık; F5 onu tekrarlıyor."*

Adım 2 — PRG ile gönder ve F5'e bas (3 dk)

Forma dönün → **"Kaydet (PRG ile)"** düğmesi.

Liste sayfasına düşersiniz, üstte ✓ mesajı var ve **adres çubuğunda** `/Kitap` yazıyor.

F5. Hiçbir şey eklenmiyor; sayfa yalnızca yeniden okunuyor.

Kodda tek satır farkı gösterin:

```
return View("EkleYanlis");          // ⚠️ adres çubuğunda POST kalır
return RedirectToAction("Index");   // ✓ PRG — adres çubuğunda GET
```

> **Sınıfa sor:** "*`ModelState` geçersizken de yönlendirmeli miyiz?*" Cevap: **hayır.** Kullanıcının düzeltmesi gereken form orada; `View(girdi)` döner.

Adım 3 — Over-posting: iki form, tek satır fark (5 dk)

`/Form/Tuzak` sayfasını açın. İki form var; **tek fark** ikincisindeki satır:

```
<input type="hidden" name="Onayli" value="true" />
```

> **Sınıfa sor:** "*Formda 'Onaylı' diye bir alan yok. Bu satırı eklersem ne olur?*"

Birinci formu gönderin → `Onayli = False`. **İkinci formu** gönderin → `Onayli = True`.

⛔ **Burada durun.** Söylenecek cümle:

> "*Kullanıcı sayfanın HTML'ini kendi makinesinde değiştirebilir. Formun HTML'i sizin, gönderilen veri değil. Bu alan `IsAdmin` ya da `Bakiye` olsaydı?*"

Sonra çözümü gösterin: `/Kitap/Ekle` action'ı parametre olarak **entity değil,** `KitapEkleVM` alıyor. VM'de `Onayli` **yok** — bağlanacak özellik olmadığı için gönderilse bile düşer. Entity'yi controller **kendisi** oluşturuyor:

```
var kitap = new Kitap { …, Onayli = false };   // ← kararı biz veriyoruz
```

Yedek plan (süre bitti): Adım 3'ü atlamayın; Adım 2'yi kısaltın (PRG'li denemeyi anlatıyla geçin, yalnızca kod satırını gösterin).

🏠 Ders Sonrası Öğrenci Görevi

1. Kendi projenizde ana varlığınız için bir **kayıt formu** kurun — **elle**, Tag Helper **kullanmadan**. Her `<input>`'un `name`'ini kendiniz yazın.
2. Formu **GET/POST action çifti** ile kurun; ikisine `[HttpGet]` / `[HttpPost]` ekleyin.
3. Formu gönderip **F12 → Network → Payload** ekran görüntüsünü alın. Gövdeyi işaretleyin: hangi anahtar hangi alandan geldi?
4. Bir alanın `name`'ini **bilerek yanlış** yazın, gönderin, sonucu ekran görüntüsüyle gösterin. Sonra düzeltin.
5. Bir sayısal alana **harf** yazıp gönderin. Parametrenin aldığı değeri ve `ModelState.IsValid`'i ekrana basıp görüntü alın.
6. POST action'ını **PRG** ile kapatın (`RedirectToAction`). Önce PRG'siz hâlinde F5 deneyin ve kayıt sayısının arttığını gösterin; sonra PRG ekleyip artmadığını gösterin.
7. Formun alabileceği alanlar için bir **ViewModel** yazın; entity'yi controller'da siz oluşturun. ViewModel'de **bilerek yer vermediğiniz** bir alanı el ile gönderip (tarayıcı geliştirici araçlarıyla ya da `curl`) **bağlanmadığını** gösterin.
8. PRG sonrası "kaydedildi" mesajını `TempData` ile taşıyın. Sonra aynı mesajı `ViewData` ile taşımayı deneyin ve yönlendirmeden sonra **görünmediğini** ekran görüntüsüyle gösterin. İki cümleyle sebebini yazın.

Teslim: proje klasörü + 8 maddenin ekran görüntüleri + **elle yazdığınız formun ürettiği HTML'i** kaydettiğiniz dosya (Hafta 5'te karşılaştıracağız) + şu soruya **iki cümlelik** yazılı cevap: "*4. maddede neden hata almadınız?*"

✅ Hocaya Kontrol Listesi

	Uygulama	Kritik an	Atlanabilir mi
A	Gövdeyi görmek	**Adım 3** — `name` ile `id` farkı	⛔ hayır
B	Kaynak sırası + Model Binding hatası	**Adım 2** (FORM kazanır) ve **Adım 4** (sessiz 0)	⛔ hayır
C	PRG + over-posting	**Adım 1** (F5 ile artan sayı) ve **Adım 3** (tek satır fark)	⛔ hayır

Her adımda sıra aynıdır: önce **tahmin ettir**, sonra **çalıştır**.

⚠️ Yedek Planların Özeti

Sorun	Yedek plan
Proje derlenmiyor	`Hafta-04/KampusKitap` bitmiş hâli hazır; canlı değişikliği anlatıya çevirin
Port meşgul	`dotnet run --urls http://localhost:5411`
F12 Network paneli okunmuyor	Yalnızca `/Form` sayfasındaki ham gövdeyle yetinin

F5 uyarısı çıkmıyor (tarayıcı sessizce tekrarlıyor)	Kayıt sayısının arttığını gösterin; uyarı kutusu tarayıcıya göre değişir
Süre taştı	Uygulama B Adım 3 (`[FromQuery]`) atlanır; slaytta anlatılır
`.cshtml` değişikliği görünmüyor	`dotnet run`'ı yeniden başlatın (Hot Reload her değişikliği yakalamaz)